

KOALA Setup Guide

JetBrains repository link: <https://github.com/JetBrains-Research/KOALA>

SEER Lab repository link: <https://github.com/seer-lab/code-tracking>

Note: IntelliJ IDEA includes a terminal but if it's too slow you can use the terminal application which is faster.

Setup

Prerequisites:

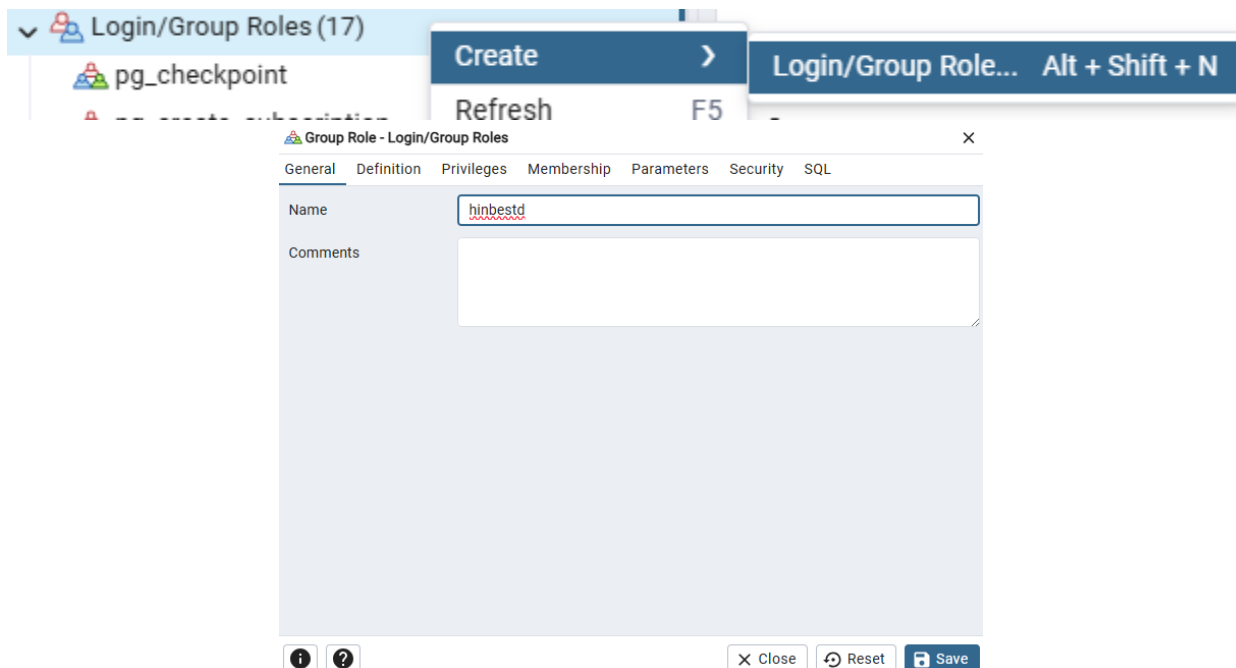
- IntelliJ IDEA
- JDK 21 or later
- Docker
- PostgreSQL
- pgAdmin or equivalent
 - o JetBrains IDEs include database tools in the Ultimate plans, which is available free to students in the JetBrains Educational Pack

PostgreSQL Database Setup

- Create a database in pgAdmin named TaskTracker

Creating a User

- This is an optional but recommended step
- Right click on **Login/Group Roles** then select **Create -> Login/Group Role...**
- A form will appear. Enter your username in the **Name** column



- Go to the **Definition** tab and enter your password

The screenshot shows the 'Group Role - Login/Group Roles' window with the 'Definition' tab selected. The window has a title bar with a close button (X) and a tab bar with 'General', 'Definition', 'Privileges', 'Membership', 'Parameters', 'Security', and 'SQL'. The 'Definition' tab contains three input fields: 'Password' with masked characters '.....', 'Account expires' with a date-time format 'YYYY-MM-DD hh:mm:ss' and a calendar icon, and 'Connection limit' with the value '-1'. A note below the 'Account expires' field states: 'Please note that if you leave this field blank, then password will never expire.' At the bottom, there are information and help icons, and buttons for 'Close', 'Reset', and 'Save'.

Field	Value
Password	
Account expires	YYYY-MM-DD hh:mm:ss
Connection limit	-1

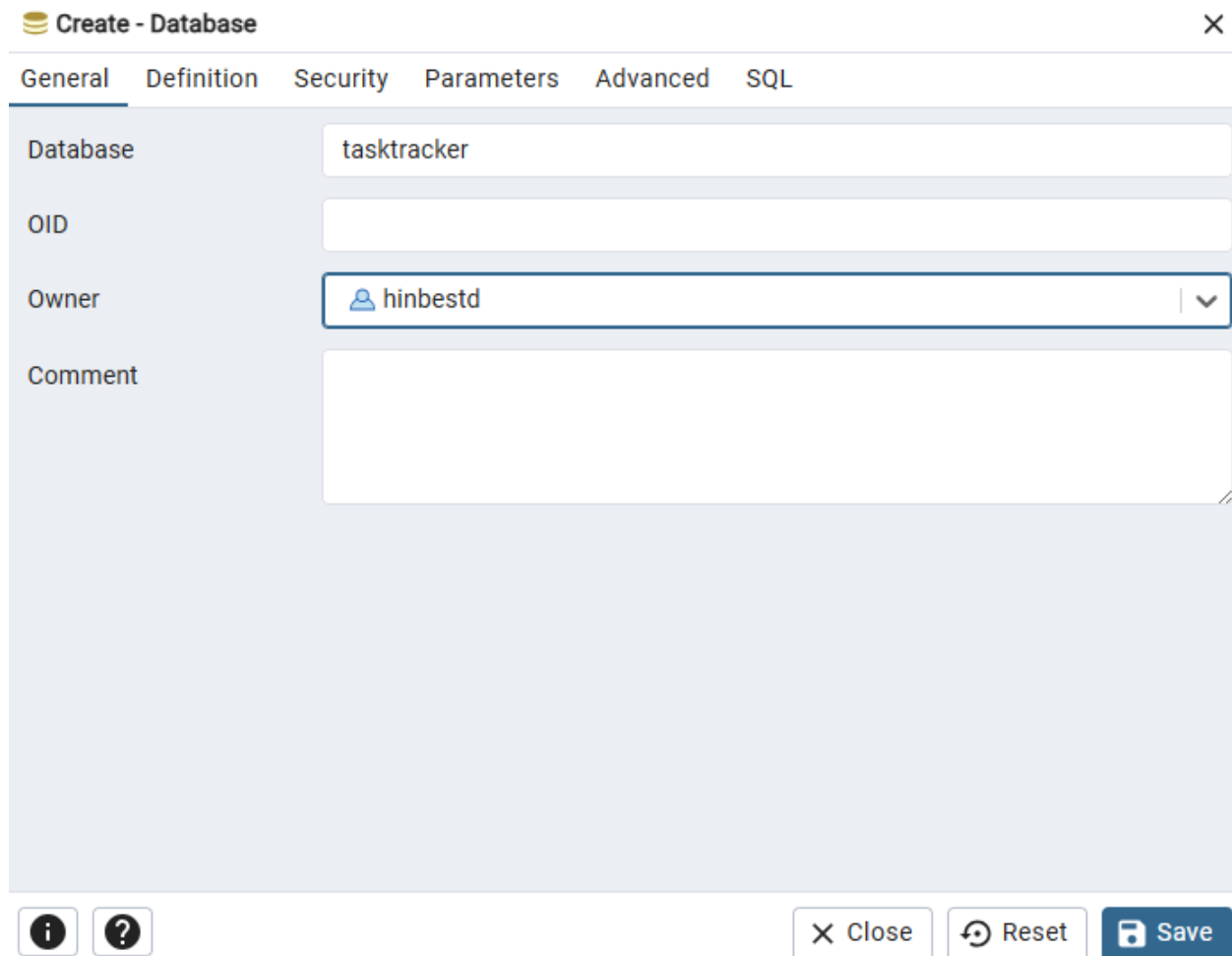
- Next, go to the **Privileges** tab and change your permissions to be able to have superuser access to the database server and click **Save**

The screenshot shows the 'Group Role - Login/Group Roles' window with the 'Privileges' tab selected. The window has a title bar with a close button (X) and a tab bar with 'General', 'Definition', 'Privileges', 'Membership', 'Parameters', 'Security', and 'SQL'. The 'Privileges' tab contains a list of permissions with toggle switches: 'Can login?' (checked), 'Superuser?' (checked), 'Create roles?' (checked), 'Create databases?' (checked), 'Inherit rights from the parent roles?' (checked), 'Can initiate streaming replication and backups?' (unchecked), and 'Bypass RLS?' (checked). At the bottom, there are information and help icons, and buttons for 'Close', 'Reset', and 'Save'.

Permission	State
Can login?	Checked
Superuser?	Checked
Create roles?	Checked
Create databases?	Checked
Inherit rights from the parent roles?	Checked
Can initiate streaming replication and backups?	Unchecked
Bypass RLS?	Checked

Creating the TaskTracker Database

- In the left-side menu, right click on **Databases** and then select **Create -> Database...** and another form will appear.
- Enter **tasktracker** into the **Database** field and assign your username to the **Owner** field and click **Save**



The screenshot shows a 'Create - Database' dialog box with the following fields and values:

Field	Value
Database	tasktracker
OID	
Owner	hinbestd
Comment	

At the bottom of the dialog, there are three buttons: 'Close', 'Reset', and 'Save'.

- Your KOALA database is now saved and ready to use. The tables will be created in the next step.

Setting up the Server

- Verify your Docker Engine is running. The server is built with Kotlin and runs on Docker
- In the root directory, open docker-compose.yml and change the DB_USERNAME and DB_PASSWORD fields to your PostgreSQL credentials.
- You can use either Option 1 or Option 2 to start your server
- Note: In both cases, after your server is built after the first setup, it will be available in the Docker app to start and stop

Option 1: Set up with the terminal

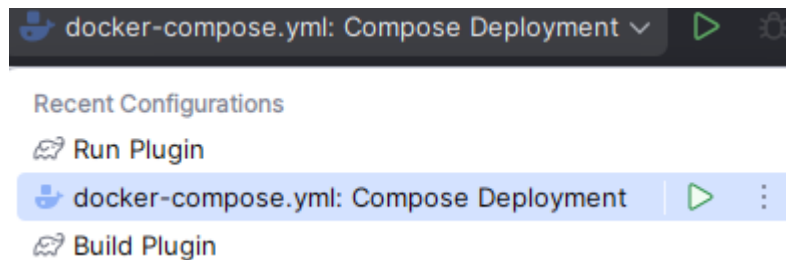
- Open the terminal on your computer or IntelliJ IDEA and open the KOALA project repository

- Run the command `docker build --platform linux/amd64 -t tasktracker-server .` to create the docker image from the Dockerfile
- Run the command `docker compose up` to create the server container (if it wasn't already created) and start the server
- Once you see the below output, your KOALA server is running properly

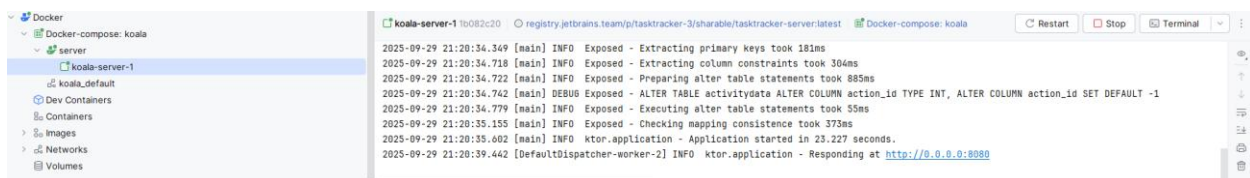
```
server-1 | 2025-09-29 21:14:37.223 [main] INFO Exposed - Executing create tables statements took 646ms
server-1 | 2025-09-29 21:14:37.528 [main] INFO Exposed - Extracting table columns took 302ms
server-1 | 2025-09-29 21:14:37.666 [main] INFO Exposed - Extracting primary keys took 136ms
server-1 | 2025-09-29 21:14:38.318 [main] INFO Exposed - Extracting column constraints took 455ms
server-1 | 2025-09-29 21:14:38.329 [main] INFO Exposed - Preparing alter table statements took 1103ms
server-1 | 2025-09-29 21:14:38.341 [main] DEBUG Exposed - ALTER TABLE activitydata ALTER COLUMN action_id TYPE INT, ALTER COLUMN action_id SET DEFAULT -1
server-1 | 2025-09-29 21:14:38.367 [main] INFO Exposed - Executing alter table statements took 35ms
server-1 | 2025-09-29 21:14:38.667 [main] INFO Exposed - Checking mapping consistence took 299ms
server-1 | 2025-09-29 21:14:39.168 [main] INFO ktor.application - Application started in 26.76 seconds.
server-1 | 2025-09-29 21:14:42.042 [DefaultDispatcher-worker-2] INFO ktor.application - Responding at http://0.0.0.0:8080
```

Option 2: Set up with IntelliJ IDEA

- At the top of the screen, click the Run Configurations menu and select **docker-compose.yml: Compose Deployment**



- Click the  button next to the configurations to start the Docker container. This will start the server.
- Once you see the following output in the console, your KOALA server is running properly.



Setting up the Plugin

- Open `gradle.properties` and verify that the `pluginSinceBuild` and `pluginUntilBuild` values are right for your release of the IntelliJ platform.
- Change the `platformType` and `platformVersion` values to the IDE and version your study is using
 - o The URLs provided in this file can guide you with this process
 - o Note: As of September 29, 2025, changing the platform version to 2025.* has caused problems but it continues to work properly as long as the build number is in line with your target build.

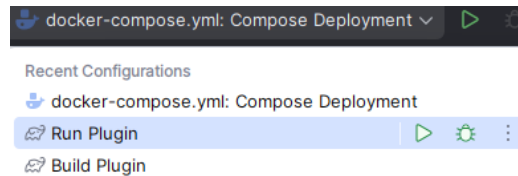
```
# Supported build number ranges and IntelliJ Platform versions -> https://plugins.jetbrains.com/docs/intellij/build-number-ranges.html
pluginSinceBuild = 243
pluginUntilBuild = 252.*

# IntelliJ Platform Properties -> https://plugins.jetbrains.com/docs/intellij/tools-gradle-intellij-plugin.html#configuration-intellij-extension
platformType = PY
platformVersion = 2024.3
```

There are two ways to run KOALA, depending on the approach that makes more sense for your situation. This includes running in debug mode (attached to the IDE) or a full build (deployment mode)

Running KOALA in Debug Mode

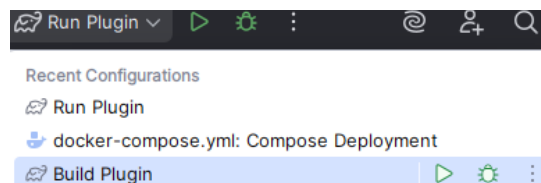
- Option 1: In the command line, run the command
`./gradlew runIde`
- Option 2: In the Run Configurations menu, select **Run Plugin** and click .



- After the build is complete, the target IDE will load and you can test out KOALA

Running KOALA in Build Mode

- Option 1: In the command line, run the command
`./gradlew :ij-plugin:buildPlugin`
- Option 2: In the Run Configurations menu, select **Build Plugin** and click .



- After the build is complete, the plugin zip file will be available in
ij-plugin/build/distributions
- **Note:** If either command is not in your run configurations, you can edit the configurations, add a new Gradle configuration and add the parameters to **Run**

Name: ☐ Store as project file

Run ... Local machine [Manage targets...](#)

Run configurations may be executed locally or on a target: for example in a Docker Container or on a remote host using SSH.

Run Modify options Alt+M

Press Alt for field hints

Gradle project:

Environment variables:

Separate variables with semicolon: VAR=value; VAR1=value1

Open run/debug tool window when started Debug Gradle scripts

Customizing KOALA

- Inside KOALA, go to `ij-plugin/src/main/resources/org/jetbrains/research/tasktracker/config`

Config Files

- View the README.md file for full instructions on using the config files

Config File	Description
Task Content	Labels the files for the experiment and sets up the task data
Info	Information about the study and the logs
Agreements	Agreements participants consent to in participating in the study
Settings and inspections	Adds restrictions or extra features for participants in the IDE
Server error message	Instructions in the event a server crash occurs
Inspection	Final page instructions after they complete the study
Survey	Add the survey to be used in the experiment
Scenario	The tasks that participants perform and the order the screens appear.